



TRAMA · INGENIERÍA DE SOFTWARE

Gobierno y Políticas

Gobierno del Equipo · Herramientas, Políticas y Lineamientos · Proyecto MANI

Documento consolidado. La **Parte I** reúne el Gobierno del Equipo: gestión del proyecto y gestión de desarrollo, incluidas las políticas técnicas y DevSecOps. La **Parte II** reúne la Matriz de Herramientas, el Tech Radar y los lineamientos derivados. Ambas partes conservan su identidad como entregables independientes y se versionan por separado.

■ Grafito ■ Verde Trama ■ Ámbar · pendientes ■ Gris · notas

Proyecto	MANI — plataforma multi-tenant de formalización de operaciones de servicio
Responsable del documento	Sara Albarracín (Scrum Master), con apoyo de Daniel Ávila (DevOps titular) para la sección 2
Vigencia	Se aprueba como línea base en el Planning que abre Sprint 1
Partes	I · Gobierno del Equipo (SM + DevOps) · II · Herramientas, Políticas y Lineamientos V1 (DevOps + Mesa de Arquitectura)
Convención	Los bloques en ámbar señalan decisiones pendientes del equipo; no son acuerdos vigentes

Contenido

0	Propósito y reglas de uso
I	PARTE I · GOBIERNO DEL EQUIPO
1	Gestión del Proyecto
1.1	Equipo, roles y autoridad de decisión
1.2	Gestión documental
1.3	Ceremonias, comunicación y priorización
1.4	Marco de medición
2	Gestión de Desarrollo
2.1	Flujo DevSecOps por etapas
2.2	Ambientes
2.3	Repositorio, ramas y Pull Requests
2.4	Security Testing, dependencias y gestión de secretos
2.5	Contenedores y despliegues
2.6	ADR y documentación técnica
2.7	Definition of Ready
2.8	Definition of Done
2.9	Gestión del incremento
II	PARTE II · HERRAMIENTAS, POLÍTICAS Y LINEAMIENTOS
3	Herramientas de proceso
4	Tecnologías de producto en evaluación
5	Tech Radar
6	Incompatibilidades, dependencias y contradicciones
7	Observación DevSecOps y Security Testing
8	Matriz por integrante y etapa DevSecOps

0. Propósito y reglas de uso

1. Lo que no está aquí, no se exige. Ninguna calificación, sanción o rechazo puede apoyarse en una regla no escrita en este documento.

- 2. Lo que está aquí, se cumple.** Las excepciones se registran como decisión explícita, con responsable y fecha, en el acta de la ceremonia correspondiente.
- 3. Este documento se modifica solo en retrospectiva,** con acuerdo del equipo, y aplica al sprint siguiente, nunca al sprint en curso ni retroactivamente.

PARTE I · GOBIERNO DEL EQUIPO

1. Gestión del Proyecto

1.1 Equipo, roles y autoridad de decisión

1.1.1 Integrantes y roles

Integrante	Rol principal	Segundo rol
Sara Albarracín	Scrum Master	Frontend
Juan Sebastián Álvarez (Alviz)	Backend	Frontend
Camila Beltrán	Frontend	Scrum Master
Nicolás Álvarez	Frontend	QA
Santiago	QA	Product Owner
Daniel Ávila	DevOps	Backend
Nicolás León	Product Owner	DevOps

Arquitecto es una responsabilidad transversal: todos los integrantes técnicos son Arquitectos y forman parte de la **Mesa de Arquitectura**, con independencia de su rol principal o secundario (ADR-0003).

PENDIENTE · PREGUNTA PARA EL EQUIPO

Titularidad DevOps: confirmar que Daniel Ávila es DevOps **titular** para toda decisión de repositorio, CI/CD y release, y que Nicolás León (DevOps secundario) apoya sin ser el titular de esas decisiones.

1.1.2 Definiciones operativas de los roles

Rol	Definición	Responsabilidad	Evidencia
Scrum Master	Facilita Scrum, promueve acuerdos, elimina impedimentos, mejora la efectividad	Reportes de sprint, seguimiento, ceremonias, gestión de Issues, uso correcto de Jira y Discord	Informe de sprint, registro de ceremonias, seguimiento de Issues, acuerdos de retro
Backend	Lógica de negocio, servicios, API, integraciones, datos	Implementa según patrones, seguridad, calidad y arquitectura aprobada	PR, pruebas, documentación de API
Frontend	Interfaz y su funcionamiento	Evalúa viabilidad de mockups, implementa componentes, cumple criterios y accesibilidad	PR, capturas y demos, pruebas de interfaz
QA	Aseguramiento de calidad, incluido Security Testing en el enfoque DevSecOps	Plan de pruebas por sprint, herramientas de prueba, verificación de criterios, registro de defectos	Plan de pruebas, casos, resultados, defectos
DevOps	Configuración técnica, automatización, CI/CD,	Repositorios, ramas, permisos, pipelines, secretos, ambientes, despliegue	Workflows, configuraciones, historial de despliegues

	ambientes; construye y ajusta la sección 2 de este documento			
Product Owner	Representa al cliente y al negocio, maximiza valor, administra el backlog; aprueba los documentos	Define y prioriza requerimientos, criterios de aceptación, Sprint Goal, valida incrementos	Backlog priorizado, historias, criterios, aprobaciones documentales	

1.1.3 Autoridad de decisión

D = decide · C = consultado antes · I = informado después

Tipo de decisión	PO	SM	DevOps	QA	Desarrollo
Prioridad y orden del backlog	D	C	I	I	I
Alcance del sprint (Planning)	D	C	C	C	C
Aceptación del incremento (Review)	D	I	I	C	I
Estimación en puntos	C	C	C	C	D
Diseño técnico y arquitectura (ADR, en la Mesa)	C	C	C	C	D*
Configuración de repositorio, CI/CD y secretos	I	C	D	I	C
Criterios de aceptación de una historia	D	C	I	C	I
Definition of Done y criterios de rechazo	C	C	C	D	C
Reglas de proceso, ceremonias y este documento	C	D	C	C	C
Umbrales de indicadores	C	D	C	C	C
Declarar un Issue bloqueante	I	D	C	C	C
Liberar a <code>main</code> (release)	C	C	D	C	I
Aprobación de documentos del proyecto	D	C	I	C	I

* La arquitectura la decide la Mesa de Arquitectura (todos los técnicos como Arquitectos), no una sola persona. El SM facilita la ceremonia; no es dueño del contenido.

Regla de aprobación documental. El PO aprueba todos los documentos del proyecto y puede comentar antes de aprobarlos. El SRS tiene además un revisor que comenta antes de la aprobación del PO.

Regla de desempate. Si dos roles con autoridad concurrente no convergen en 24 horas, la decisión escala al PO si es de producto y al SM si es de proceso. La decisión escalada se registra en `#general` con una línea: decisión, motivo y fecha.

1.2 Gestión documental

1.2.1 Distribución de la documentación

Herramienta	Qué contiene
GitHub	Código, elementos técnicos de desarrollo, ADR en <code>.md</code> , webhooks de CI, C4 y <code>workspace.dsl</code>
OneDrive	

SRS, Análisis de Requerimientos, este documento de Gobierno del Equipo, Plan de Tareas, Actas, documentación funcional y documentos administrativos y académicos. Toda hoja de cálculo, incluidas las de QA, se trabaja en Microsoft Excel y se almacena aquí

Jira	Gestión y seguimiento del trabajo (backlog, sprints, tablero) para PO y SM. El Scrum Master no usa GitHub como herramienta de gestión de su rol
-------------	---

Google Sheets no hace parte del esquema documental del proyecto.

PENDIENTE · NOTA DE SCRUM PARA QA

Para mantener una única fuente documental y evitar dispersión de información, las matrices, registros y artefactos tabulares de QA deberán trabajarse en Microsoft Excel y almacenarse en OneDrive.

1.2.2 Qué se documenta, dónde y quién

Documento	Ubicación	Responsable	Cuándo se actualiza
Gobierno del Equipo (este documento)	OneDrive	Scrum Master (§1) + DevOps (§2)	En retrospectiva
Instructivo de configuración	GitHub /docs	DevOps	Al cambiar la configuración
Modelo C4	GitHub /docs/c4	Mesa de Arquitectura (rotativo)	Al cambiar contenedores o componentes
ADR	GitHub /docs/adr/ADR-NNNN	Redactor rotativo de la Mesa	Al tomar la decisión, nunca después
Estrategia y matrices de pruebas	OneDrive (Excel)	QA	Al cambiar niveles o herramientas
Análisis de Requerimientos	OneDrive	PO	Al cambiar necesidades o requerimientos
SRS	OneDrive	PO elabora · revisor comenta · PO aprueba	Al cambiar requisitos
Product Backlog	Jira	PO	Continuo
Plan de Tareas	OneDrive	Scrum Master	Por sprint
Informe de sprint	OneDrive (Excel)	Scrum Master	Al cierre de cada sprint

1.3 Ceremonias, comunicación y priorización

1.3.1 Espacios por ceremonia

Canal o espacio	Propósito	Regla
#daily	Reporte diario escrito y registro de Issues, incluidos impedimentos y bloqueos reales	Formato en §1.3.2
#mesa-arquitectura	Convocatorias, fichas y actas de la Mesa de Arquitectura	Toda decisión produce ADR
#sprint-planning	Convocatoria y CHECK-IN del Sprint Planning	Solo el SM publica CHECK-IN

#retro-back	Reporte escrito de retrospectiva y CHECK-IN	Solo el SM publica CHECK-IN
#desarrollo , #qa , #devops , #gestion-de-proyectos	Discusión técnica por área	Las conclusiones se suben al issue o al ADR
#github-actividad , #github-ci	Eventos de issues, PR y CI vía webhook	Solo lectura; los humanos no escriben

Regla de canal único de verdad. Discord y Jira coordinan; no son repositorio de decisiones. Toda conclusión que afecte código o alcance se escribe en el issue (Jira) o en un ADR (GitHub). Lo que solo existe en un chat, no existe.

1.3.2 Formato de reportes escritos

Daily — un mensaje por persona en #daily :

- **Trabajo realizado:** qué terminé desde el último reporte, con referencia al issue.
- **Trabajo siguiente:** en qué trabajaré hasta el próximo daily, con issue.
- **Trabajo compartido:** menciono explícitamente a quién o quiénes, si trabajé acompañado.
- **Issues:** issues abiertos o en curso que me afectan.
- **Impedimentos o bloqueos reales:** ninguno, o descripción del bloqueo y a quién necesito.

Un mensaje sin *Trabajo realizado*, *Trabajo siguiente* e *Issues/Impedimentos* no cuenta para el indicador de cumplimiento del daily.

Retrospectiva — un mensaje por persona con cuatro bloques: *Bien*, *Mejorar*, *Aprendizaje*, *Duda*. Debe contener los cuatro para ser válido.

1.3.3 Ceremonias

Ceremonia	Frecuencia	Duración	Convoca	Asistencia
Sprint Planning	Inicio de sprint	2 h (virtual)	SM + PO	Todo el equipo
Daily	Diaria (hábiles)	Reporte escrito	SM	Todo el equipo
Sprint Review	Cierre de sprint	1 h (en clase, con el profesor)	PO	Todo el equipo + profesor
Retrospectiva	Cierre de sprint	1 h + reporte escrito	SM	Todo el equipo
Mesa de Arquitectura	Según necesidad	~1 h	SM (facilita)	Todo el equipo (Arquitectos)

Mesa de Arquitectura. Preparación con ficha 24 h antes y mínimo dos alternativas reales; quórum de 5 de 7; 20 min por decisión; un integrante argumenta en contra; ADR numerado por decisión, con redactor rotativo distinto al proponente; el disenso queda documentado. Ningún acuerdo se aprueba sin alternativas ni sin el requisito que lo justifica.

El CHECK-IN se habilita al inicio de la ceremonia y se cierra al finalizar. Registrar asistencia sin participar se considera alteración del registro y se revisa en retrospectiva.

1.3.4 Tiempos de respuesta y gestión de Issues

Situación	Plazo
Mención directa en canal de trabajo	6 horas hábiles
Solicitud de revisión de PR	24 horas hábiles
Issue con impedimento o bloqueo	Reconocimiento del SM ≤ 4 h hábiles; respuesta del involucrado ≤ 6 h hábiles
Fallo de CI en <code>develoP</code>	Atención inmediata (< 4 h) del autor del último merge

Terminología: Impediments → Issues. La gestión general se centraliza en Issues. Cuando exista un bloqueo real, se reporta en el bloque correspondiente del daily y se registra como Issue con causa, impacto, responsable requerido y acción esperada.

1.3.5 Priorización y estimación

- **Priorización del backlog:** decide el PO, consultando al SM; escala de prioridad en el Product Backlog (Crítica, Alta, Media, Baja).
- **Estimación:** decide el equipo de Desarrollo, en puntos de historia (Fibonacci); sin consenso, se resuelve por Planning Poker facilitado por el SM.

1.3.6 Cohesión y bienestar del equipo

Mecanismo sencillo y no invasivo de seguimiento a la salud y cohesión del equipo: cohesión, comunicación, colaboración, carga percibida, bienestar durante el sprint e identificación temprana de factores humanos que puedan afectar el trabajo colectivo. No mide el ánimo ni afecta la nota; alimenta las acciones de mejora de la retrospectiva.

1.4 Marco de medición

1.4.1 Composición de la nota

TOTAL = $0,60 \times \text{Nota de equipo} + 0,40 \times \text{Nota individual de proceso}$

Escala 1-5: 1 insuficiente · 2 por debajo · 3 cumple · 4 supera · 5 referente.

1.4.2 Fuentes de datos

Los insumos provienen de Jira, GitHub, GitHub Actions y Discord, y de los bots definidos. No se modifican manualmente sin justificación y trazabilidad interna del propio indicador. Cuando exista ajuste manual, el informe conserva el valor original, el ajustado, la causa, quién autoriza y la fecha.

PENDIENTE · PROPUESTA PARA DEVOPS Y SM

Validar antes del cierre de Sprint 1 que los indicadores se puedan extraer desde Jira, o desde Jira y GitHub.

1.4.3 Indicadores de equipo (60 %)

#	Indicador	Fórmula	Meta	Peso
E1	Predictibilidad del sprint	$\text{completado} \div \text{comprometido} \times 100$	85-110 %	0,25

E2	Estabilidad del flujo (CFD)	días sin superar WIP ÷ días hábiles × 100	≥ 85 %	0,20
E3	Cycle time p85	percentil 85 In Progress → Done	≤ sprint anterior	0,15
E4	Aceptación sin retrabajo	aceptados sin devolución ÷ entregados × 100	≥ 85 %	0,15
E5	Tasa de retrabajo	horas de corrección ÷ horas totales × 100	≤ 15 %	0,15
E6	Defectos escapados	bugs tras aceptación o liberación	≤ 2	0,10

E1 se evalúa por la desviación en puntos porcentuales respecto a 100 %. Nota de equipo ≥ 4,0.

1.4.4 Indicadores individuales de proceso (40 %)

#	Indicador	Fórmula	Fuente	Peso
I1	Fiabilidad del compromiso	puntos propios Done ÷ puntos propios comprometidos	Jira	0,30
I2	Participación en revisión de código	revisiones emitidas ÷ PR de otros abiertos	GitHub	0,25
I3	Cumplimiento de la DoD	elementos propios que cumplen DoD ÷ presentados como terminados	Jira + QA	0,20
I4	Trazabilidad de Issues y bloqueos	issues bien documentados y gestionados ÷ total × 100	Canal + issue	0,15
I5	Asistencia a ceremonias	asistidas ÷ programadas	Discord (CHECK-IN)	0,10

Ajuste por rol. PO y SM, en su rol principal, no producen código de forma obligatoria y no generan datos comparables para I1 ni I3; su nota se calcula sobre I2, I4 e I5, renormalizando pesos. Cuando ejercen un segundo rol técnico y asumen desarrollo, los indicadores técnicos aplican solo a esos elementos.

1.4.5 Umbrales de KPIs

Equipo

Indicador	5	4	3	2	1
E1 Predictibilidad — desviación r-100	≤ 10 pp	≤ 20 pp	≤ 30 pp	≤ 45 pp	> 45 pp
E2 Estabilidad (CFD)	≥ 95 %	≥ 90 %	≥ 85 %	≥ 70 %	< 70 %
E3 Cycle time p85	≤ 3 d	≤ 5 d	≤ 8 d	≤ 12 d	> 12 d
E4 Aceptación sin retrabajo	≥ 95 %	≥ 85 %	≥ 75 %	≥ 60 %	< 60 %
E5 Tasa de retrabajo	≤ 5 %	≤ 10 %	≤ 15 %	≤ 25 %	> 25 %
E6 Defectos escapados	0	1	2	3-4	≥ 5

Individuales

Indicador	5	4	3	2	1
I1 Fiabilidad	≥ 95 %	≥ 85 %	≥ 70 %	≥ 50 %	< 50 %
I2 Revisión de código	≥ 0,80	≥ 0,60	≥ 0,40	≥ 0,20	< 0,20

I3 Cumplimiento DoD	≥ 95 %	≥ 85 %	≥ 75 %	≥ 60 %	< 60 %
I4 Trazabilidad (Issues)	100 %	≥ 90 %	≥ 75 %	≥ 50 %	< 50 %
I5 Asistencia	100 %	≥ 90 %	≥ 75 %	≥ 60 %	< 60 %

El Sprint 1 funciona como línea base; la recalibración se aprueba en retrospectiva y aplica desde el sprint siguiente.

1.4.6 Estado general del sprint

Estado	Criterio
Verde	Carry-over ≤ 10 %, Sprint Goal cumplido, sin bloqueo crítico sin gestión
Amarillo	Carry-over > 10 % y ≤ 25 %, o Sprint Goal en riesgo (definir acciones)
Rojo	Carry-over > 25 %, Sprint Goal incumplido, bloqueo crítico no resuelto o trabajo trasladado durante tres o más sprints

Control de cambios. Toda modificación de roles, umbrales, fórmulas, fuentes o criterios se registra en retrospectiva con fecha, responsables y sprint de entrada en vigencia.

2. Gestión de Desarrollo

Políticas técnicas y de DevSecOps. Responsable de construcción y ajuste: Daniel Ávila (DevOps titular), con apoyo de Nicolás León (DevOps secundario).

2.1 Flujo DevSecOps por etapas

Etapa	Actividad	Responsable	Herramienta y estado
Plan	Backlog, sprint, tablero	PO / SM	Jira (adoptada)
Code	Desarrollo backend y frontend	Backend / Frontend	Stack en evaluación (ver Matriz de Herramientas)
Build	Compilación y empaquetado	DevOps	Docker / Compose (probar); GitHub Actions
Test	Pruebas funcionales y de carga	QA	Postman + Newman (funcional), k6 (carga)
Security	Security Testing	QA + DevOps	Sin herramienta definida (ver §2.4)
Release	Liberación a <code>main</code>	DevOps titular	GitHub Actions
Deploy	Despliegue a ambientes	DevOps	Docker; Kubernetes en evaluación
Operate / Monitor	Operación y observabilidad	DevOps	OpenTelemetry (evaluar)

2.2 Ambientes

Ambientes contemplados: **DEV, QA y PROD**. Configuración y promoción entre ambientes a cargo de DevOps.

PENDIENTE · PROPUESTA PARA LA MESA DE ARQUITECTURA

Definir el driver de orquestación y escalado antes de decidir sobre Kubernetes. Docker Compose no realiza autoescalado; si el atributo de calidad lo exige, se evalúa Kubernetes con su ADR cuando la Mesa lo decida.

2.3 Repositorio, ramas y Pull Requests

- **Ramas:** `main` (release), `develop` (integración), `feature/US-xx-descripcion`, `fix/...`, `spike/SP-xx-...`.
- **Pull Request:** todo cambio entra por PR aprobado, con CI en verde y al menos una revisión de otro integrante.
- Un fallo de CI en `develop` se atiende de inmediato (< 4 h) por el autor del último merge.
- **Liberación a `main`:** la decide DevOps titular.

PENDIENTE · PROPUESTA PARA DEVOPS

Confirmar la nomenclatura exacta de ramas antes del Sprint 1, dado que la DoD exige que la rama la cumpla.

2.4 Security Testing, dependencias y gestión de secretos

El enfoque del proyecto es DevSecOps y QA incluye Security Testing, pero no hay herramienta definida todavía. Cobertura mínima requerida:

- **SAST** — análisis estático de código.
- **Análisis de dependencias** — vulnerabilidades en librerías.
- **Gestión de secretos** — no exponer credenciales en el repositorio ni en pipelines.

PENDIENTE · PREGUNTA PARA QA, DEVOPS Y LA MESA

¿Qué se usará para cada una de estas tres coberturas? Queda como evaluación pendiente; ninguna herramienta se cierra aún.

2.5 Contenedores y despliegues

Contenerización de ambientes DEV y QA con Docker y Docker Compose. Orquestación y escalado (Kubernetes) permanecen en evaluación (§2.2). Ningún despliegue a PROD ocurre sin pasar por DEV y QA con CI en verde.

2.6 ADR y documentación técnica

- Toda decisión técnica costosa de revertir se registra como ADR **antes** de implementarse, con contexto, alternativas, decisión, trade-off y consecuencias.
- Un ADR no se borra ni se edita: si la decisión cambia, se escribe uno nuevo que declare `supersedes ADR-NNNN`.
- Los ADR viven en GitHub `/docs/adr/` en formato `.md`; la autoría rota entre integrantes en cada Mesa.
- Ningún ADR se anticipa antes de que la Mesa tome la decisión correspondiente.

2.7 Definition of Ready

Una historia entra al Planning solo si tiene criterios de aceptación en BDD, prioridad, clase, estimación propuesta y ningún spike bloqueante abierto.

PENDIENTE · PROPUESTA PARA EL EQUIPO

Adoptar esta Definition of Ready desde el Sprint 1.

2.8 Definition of Done

Ningún elemento es Done con pruebas pendientes, documentación incompleta, defectos abiertos o criterios incumplidos.

Dimensión	Criterio
Funcionales	Criterios verificados y aceptación del PO cuando corresponda
Técnicos	Integrado por PR aprobado, rama con nomenclatura correcta y CI en verde
De calidad	QA ejecuta y valida pruebas; sin defectos abiertos que impidan la entrega
Documentales	Documentación técnica, de API, de usuario y de arquitectura actualizada cuando el cambio lo exige

De gestión Issue, PR, pruebas y aceptación vinculados en Jira

2.9 Gestión del incremento

Concepto	Definición y tratamiento
Incremento entregado	Trabajo integrado que cumple la DoD, validado y liberable. Evidencia: ítem en estado Done, fecha, PR, evidencias de prueba y aceptación del PO
Incremento no entregado	Comprometido en Planning que no llegó a Done. El responsable documenta la causa, el SM registra el impacto y el PO re-prioriza. No cuenta como parcial
Sprint destino	Sprint futuro al que se reasigna lo no completado, analizado en retrospectiva y priorizado por el PO en el Planning siguiente

PARTE II · HERRAMIENTAS, POLÍTICAS Y LINEAMIENTOS

3. Herramientas de proceso

Estado del documento: en construcción. Esta parte distingue dos naturalezas de herramienta: las de **proceso**, que pueden cerrarse cuando hay decisión del equipo, y las **tecnologías de producto**, que no se adoptan sin ADR. Ninguna tecnología de producto está adoptada hasta que exista su ADR.

Herramientas de gestión, seguimiento, documentación, control de código, CI/CD, pruebas, calidad y colaboración.

Herramienta	Uso	Justificación	Estado
Jira	Gestión y seguimiento del trabajo (backlog, sprints, tablero) para PO y SM	Herramienta única de gestión definida en ADR-0002	Adoptada
GitHub	Código, elementos técnicos, ADR en <code>.md</code> , webhooks de CI, C4 y <code>workspace.dsl</code>	Repositorio de código y ADR (ADR-0001, ADR-0002)	Adoptada
OneDrive	Documentación formal: SRS, Requerimientos, Gobierno del Equipo, Plan de Tareas, Actas	Fuente de verdad de documentos formales (ADR-0001)	Adoptada
Discord	Coordinación y ceremonias, con canales por ceremonia	Medio de coordinación; no es repositorio de decisiones	Adoptada
GitHub Actions	CI/CD: pipelines, build y checks	Integración continua sobre el repositorio	Adoptada
Postman + Newman	Pruebas funcionales de API, con Newman en CI	Verificación funcional de contratos de API	Adoptada
k6	Pruebas de rendimiento y carga	Verifica concurrencia en despacho y mensajería	Adoptada
Figma	Diseño de interfaz y mockups	Insumo de Frontend; validación de viabilidad técnica	Adoptada
Miro / Mermaid	Diagramas de flujo y blueprints en fase de análisis	Apoyo a requerimientos y diseño	Adoptada

3.1 Herramientas evaluadas y no adoptadas

Herramienta	Motivo
GitLab Issues	No se adopta (ADR-0002). El rastreador del proyecto es Jira
Confluence	No se adopta. La documentación formal vive en OneDrive
Google Sheets	No aplica para QA ni para ningún artefacto del proyecto. Toda hoja de cálculo, incluidas las matrices y registros de QA, se trabaja en Microsoft Excel y se almacena en OneDrive (§1.2.1)

4. Tecnologías de producto en evaluación

Stack con el que eventualmente se construirá el producto. **Ninguna está adoptada**, y ninguna pasa a «Adoptar» ni recibe número de ADR hasta que la Mesa de Arquitectura tome la decisión correspondiente.

Tecnología	Propósito propuesto	Anillo	Spike relacionado
NestJS	Framework backend	Evaluar	SP-01.1.1 (ligado a persistencia y backend)
MongoDB	Motor de persistencia documental	Evaluar	SP-01.1.1
Supabase	Backend-as-a-service sobre PostgreSQL: auth, RLS, storage	Evaluar	SP-01.1.1, SP-01.2.1
Flutter + Dart	Aplicación cliente multiplataforma	Evaluar	Sin spike de frontend asignado
Kubernetes	Orquestación de contenedores y escalado	Evaluar	Pendiente definir el driver (\$5)
Docker / Compose	Contenerización de ambientes DEV y QA	Probar	—
OpenTelemetry	Observabilidad	Evaluar	Sin spike de observabilidad asignado

5. Tech Radar

PENDIENTE · PROPUESTA PARA DEVOPS Y LA MESA DE ARQUITECTURA

El Tech Radar, entendido como mecanismo permanente de clasificación tecnológica en cuatro anillos, **todavía no es una decisión tomada y no tiene ADR**. Corresponde a DevOps (Daniel Ávila, titular) llevar la propuesta a la Mesa de Arquitectura. Mientras tanto, el equipo usa los anillos de forma informal, solo como clasificación de trabajo.

Anillo	Significado	Contenido actual
Adoptar	Decisión tomada y respaldada por ADR	Hoy vacío en tecnologías de producto
Probar	Se usará en un spike o PoC controlado	Docker / Docker Compose
Evaluar	Candidata; requiere spike y ADR antes de avanzar	NestJS, MongoDB, Supabase, Flutter, Kubernetes, OpenTelemetry
Contener	Evitar o limitar su expansión	Sin elementos registrados

Regla vigente del equipo (aún no formalizada como ADR): una tecnología no pasa a «Adoptar» hasta que exista el ADR correspondiente.

6. Incompatibilidades, dependencias y contradicciones

Se registran aquí, no dentro de los requerimientos ni de las historias.

#	Situación	Estado
1	MongoDB (documental, sin RLS nativo) frente a Row-Level Security y Supabase (PostgreSQL). El spike SP-01.1.1 evalúa RLS como estrategia de aislamiento multi-tenant. RLS es propio de bases relacionales, de modo que elegir MongoDB excluye RLS como mecanismo de aislamiento. No pueden coexistir como decisión primaria	Abierta · la resuelve la Mesa a partir de SP-01.1.1
2		Abierta · depende del punto 1

El plan de QA asume Supabase + RLS mientras Backend propuso MongoDB. Es la misma contradicción del punto 1, aflorando en las herramientas de prueba. El plan de pruebas debe esperar la decisión de persistencia antes de fijar la verificación de RLS

3	Tres rastreadores propuestos (GitHub Projects, GitLab Issues, Jira)	Resuelta por ADR-0002: Jira para gestión, GitHub para lo técnico y los ADR, GitLab descartado
4	Docker Compose frente a escalado automático. Docker Compose no realiza autoescalado; la tecnología en evaluación para orquestación y escalado es Kubernetes	Abierta · sin driver de calidad que justifique Kubernetes, sería sobrearquitectura
5	Confluence y Jira como documentación (propuesta del PO) frente a OneDrive y GitHub	Resuelta por ADR-0001 y ADR-0002: OneDrive para documentos formales, GitHub para ADR y código, Confluence descartado
6	Herramientas de diagramación (Miro, Mermaid, Figma) frente al modelo C4. Para el SDD se usará un framework de modelado, C4 o 4+1	Abierta · ¿la herramienta de C4 se decide en la fase de diseño? No corresponde a este sprint

7. Observación DevSecOps y Security Testing

El proyecto establece un enfoque **DevSecOps** y las actividades de QA incluyen Security Testing, pero **actualmente no existen herramientas de seguridad propuestas**. Postman cubre pruebas funcionales de API y k6 pruebas de carga; ninguna resuelve por sí sola el Security Testing.

PENDIENTE · PREGUNTA PARA QA, DEVOPS Y LA MESA DE ARQUITECTURA

¿Qué herramientas o mecanismos cubrirán, como mínimo, el análisis estático de código (SAST), el análisis de dependencias y la gestión de secretos? No se selecciona ninguna herramienta todavía: el ADR correspondiente se crea únicamente cuando la Mesa tome la decisión. Esta pendiente es la misma que se registra en §2.4 de la Parte I.

8. Matriz por integrante y etapa DevSecOps

Estado actual, en evaluación.

Integrante	Rol	Etapas DevSecOps	Herramienta propuesta	Naturaleza
Sara Albarracín	Scrum Master	Plan	Jira + OneDrive	Proceso · adoptada
Juan Sebastián Álvarez (Alviz)	Backend	Code / Build	NestJS + MongoDB	Producto · Evaluar
Camila Beltrán	Frontend	Code / Build	Flutter + Dart (+ Figma)	Producto · Evaluar
Nicolás Álvarez	Frontend	Code / Build	Flutter + Dart	Producto · Evaluar
Santiago	QA	Test / Security Testing	Postman + Newman + k6 + emulador	Proceso · adoptada; falta Security Testing
Daniel Ávila	DevOps	Release / Deploy / Operate / Monitor	GitHub Actions + Docker/Compose + OpenTelemetry (+ Kubernetes)	Mixto · proceso adoptado; orquestación en evaluación

Nicolás León	Product Owner	Requirements / Mockups	Jira + Figma + Miro + Mermaid	Proceso · adoptada
--------------	---------------	------------------------	-------------------------------	--------------------